

LaunchVector for Students — Enterprise Solution Architecture

Document Type: Production-Ready Solution Architecture

Version: 1.0

Classification: Confidential — Technical Design

Date: May 2026

Prepared by: Enterprise Architecture Office

Table of Contents

1. [Executive Summary](#)
 2. [Business Objectives & Problem Statement](#)
 3. [Requirements Analysis](#)
 4. [High-Level Architecture Overview](#)
 5. [Component Design & Data Flow](#)
 6. [Technology Stack](#)
 7. [Cloud Infrastructure & Networking](#)
 8. [Security Architecture](#)
 9. [Data Architecture & Storage](#)
 10. [API Design & Integration Layer](#)
 11. [AI/ML Pipeline Architecture](#)
 12. [Scalability & Performance](#)
 13. [Reliability, Fault Tolerance & DR](#)
 14. [Observability & Monitoring](#)
 15. [CI/CD & DevOps Pipeline](#)
 16. [Deployment Strategy & Environment Separation](#)
 17. [Third-Party Integrations](#)
 18. [Cost Architecture & Optimisation](#)
 19. [Architectural Tradeoffs & Risk Register](#)
 20. [Phased Implementation Roadmap](#)
-

1. Executive Summary

LaunchVector for Students is an AI-native career intelligence platform serving the 4M+ annual cohort of fresh graduates and final-year students in the US and UK. It autonomously applies to

jobs on behalf of students, tailors CVs per job description, identifies graduate scheme deadlines, coaches interview responses via AI voice simulation, and learns from every application outcome to improve match quality over time.

The platform must serve a price-sensitive, mobile-first, high-volume user base (\$29/month) while simultaneously supporting institutional B2B contracts with universities and career offices (\$50K-\$250K/year). The architecture must therefore be:

- **Multi-tenant** across B2C and B2B customer segments with strict data isolation
- **AI-first** with LLM pipelines, browser automation agents, and outcome-learning ML models as first-class architectural components
- **Mobile-optimised** with offline-capable PWA and React Native clients
- **Globally distributed** with sub-200ms latency for job matching and real-time application status
- **Cost-elastic** — AI compute costs must scale proportionally to revenue, not ahead of it

Primary cloud: AWS (primary region us-east-1; secondary eu-west-2 for UK compliance)

Architecture pattern: Event-driven microservices with CQRS for the application engine; serverless for stateless AI tasks; edge caching for the job intelligence feed.

2. Business Objectives & Problem Statement

2.1 Business Objectives

Objective	Measurable Target
Serve graduate job seekers at disruptive price point	\$29/month vs \$2K-\$15K for human services
Automate 80–85% of job search labor	20 tailored ATS submissions/week per student
Deliver measurable outcome improvement	>15% application response rate vs 5% industry baseline
Scale to 50K+ concurrent active users by Month 12	Without proportional cost increase
Support university B2B institutional contracts	Multi-tenant data isolation; white-label capability
Build proprietary outcome-learning data moat	1M+ application records by end of Year 1

2.2 Core Technical Problems

1. **Browser automation at scale:** Submitting forms across 50+ heterogeneous ATS platforms (Workday, Greenhouse, Lever, Ashby, Rippling) requires resilient, maintainable browser agents that handle CAPTCHAs, rate limits, and UI drift without human intervention.
2. **LLM cost at volume:** Each student generates 20 resume-tailoring LLM calls per week. At 10K active users, that is 200K LLM completions per week. Prompt caching, batching, and model tiering are architectural requirements, not optimisations.
3. **Real-time pipeline state:** Students need live visibility of application status (submitted → viewed → response → interview). This requires event streaming, webhook ingestion from multiple employer systems, and a low-latency read model.
4. **Multi-tenant B2B isolation:** University contracts require strict data segregation, custom branding, SSO via institutional identity providers, and audit logs — while sharing the underlying platform infrastructure for cost efficiency.
5. **Outcome learning feedback loop:** Application outcome data (interview invite rate, offer rate by role/company/resume version) must feed back into the matching and tailoring models with a latency of hours, not days.

2.3 Architectural Assumptions

The following are explicitly stated assumptions made in the absence of detailed requirements:

- Primary deployment cloud is **AWS**; UK data residency handled via eu-west-2 secondary region
 - Students are primarily mobile users (iOS/Android + PWA); desktop is secondary
 - University B2B contracts use **SAML 2.0 or OIDC** for SSO via institutional IdPs (Shibboleth, Azure AD)
 - The platform does **not** directly access employer ATS APIs (most are not publicly available); browser automation is the primary submission mechanism
 - LinkedIn automation is out of scope for MVP due to ToS risk; email-based outreach is in scope from day one
 - Voice interview AI uses a third-party voice synthesis and STT provider (ElevenLabs + Deepgram), not a custom model
 - GDPR and FERPA compliance are mandatory from launch (EU/UK users and US university data)
 - The outcome-learning ML model is a fine-tuned ranker, not a foundation model; training runs weekly on accumulated data
-

3. Requirements Analysis

3.1 Functional Requirements

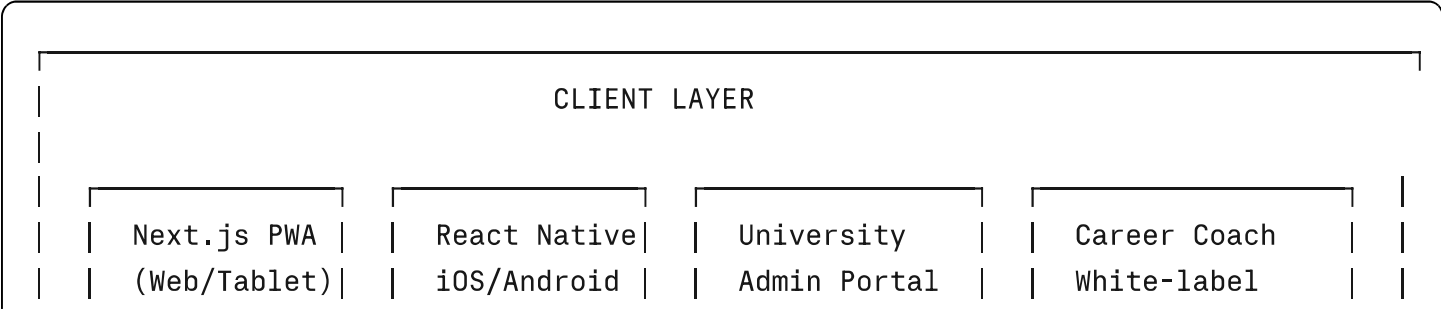
Domain	Requirement	Priority
Onboarding	CV upload (PDF/DOCX) → structured profile extraction	P0
Onboarding	Conversational goal-setting intake (roles, salary, location, preferences)	P0
Onboarding	AI gap analysis: CV scored vs target JDs	P0
Application Engine	ATS form completion via browser agent across 20+ platforms (MVP: 5)	P0
Application Engine	Per-JD resume tailoring using LLM	P0
Application Engine	Cover letter generation per application	P0
Application Engine	Application status dashboard (real-time pipeline)	P0
Job Intelligence	Job feed with match score, competition level, salary data	P0
Job Intelligence	Graduate scheme deadline detection and early-application alerts	P1
Outreach	Hiring manager outreach drafting and sending (email)	P1
Outreach	Follow-up sequence automation (7-day, 14-day)	P1
Interview Prep	Company research brief auto-generation	P1
Interview Prep	AI voice mock interview with STAR feedback	P1
Salary Intelligence	Real-time market benchmarks per role/location	P2
Outcome Learning	Application outcome recording and model retraining pipeline	P1
B2B/University	White-label dashboard for institution	P1
B2B/University	SSO via institutional IdP (SAML/OIDC)	P1
B2B/University	Bulk student cohort management and reporting	P2
Billing	Stripe subscription management (monthly, annual)	P0
Billing	Usage metering for API tier	P2

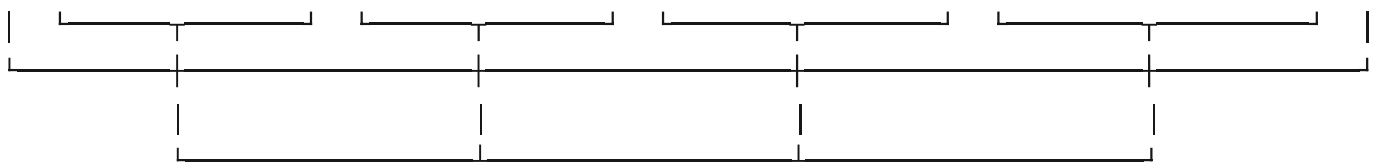
3.2 Non-Functional Requirements

Category	Requirement	Target
Availability	Platform uptime (excl. planned maintenance)	99.9% (8.7 hrs downtime/year)
Latency	Job feed load (cached)	< 150ms p99
Latency	CV parse and profile extraction	< 30s
Latency	ATS form submission per job	< 3 min per application
Throughput	Concurrent browser automation workers	500+ (auto-scaling)
Scale	Registered users (Year 1 target)	50,000
Scale	Concurrent active application sessions	5,000
Security	Data encryption (at rest, in transit)	AES-256 / TLS 1.3
Compliance	GDPR (EU/UK), FERPA (US university data)	Mandatory from launch
Compliance	SOC 2 Type II	Target within 18 months
RTO	Recovery Time Objective (production outage)	< 1 hour
RPO	Recovery Point Objective (data loss window)	< 15 minutes
Maintainability	Service deployment independence	Per-service CI/CD
Cost	LLM compute cost as % of ARPU	< 30% (\$8.70/user/month)
Observability	Distributed trace coverage	> 95% of request paths

4. High-Level Architecture Overview

4.1 Architecture Diagram





| HTTPS/WSS

EDGE & API GATEWAY LAYER

| CloudFront CDN (static assets, API response caching)

| AWS API Gateway (REST) + AWS API Gateway (WebSocket)

| |— Rate limiting (per-user, per-tenant, per-IP)

| |— JWT validation (Cognito authorizer)

| |— Request routing to microservices

| |— WAF (OWASP Top 10, DDoS protection)

CORE MICROSERVICES LAYER (EKS)

| Identity
| Service

- Auth/SSO
- MFA
- Tenancy
- RBAC

| Profile
| Service

- CV parse
- Gap score
- Resume tailoring

| Job
| Intelligence
| Service

- Job feed
- Matching
- Deadlines
- Salary

| Application
| Orchestrator
| Service

- Workflow mgmt
- Agent dispatch
- Status tracking
- Retry logic

▼	▼	▼	▼
Outreach Service	Interview AI Service	Billing Service	Notification Service
- Email drafting	- Mock ivw	- Stripe	- Push/Email/SMS
- Follow-up sequences	- Voice AI	- Metering	- WebSocket push
	- STAR eval	- Invoicing	- Digest scheduler
	- Co briefs		

ASYNC PROCESSING LAYER

Amazon SQS + SNS (Event Bus)

Topics: application.submitted | job.matched | outcome.recorded

outreach.scheduled | interview.scheduled | webhook.received

WORKER FLEET (EKS Node Group – GPU-capable for ML)

Browser Agent

LLM Pipeline

ML Training

Workers

Workers

Workers

(Playwright)

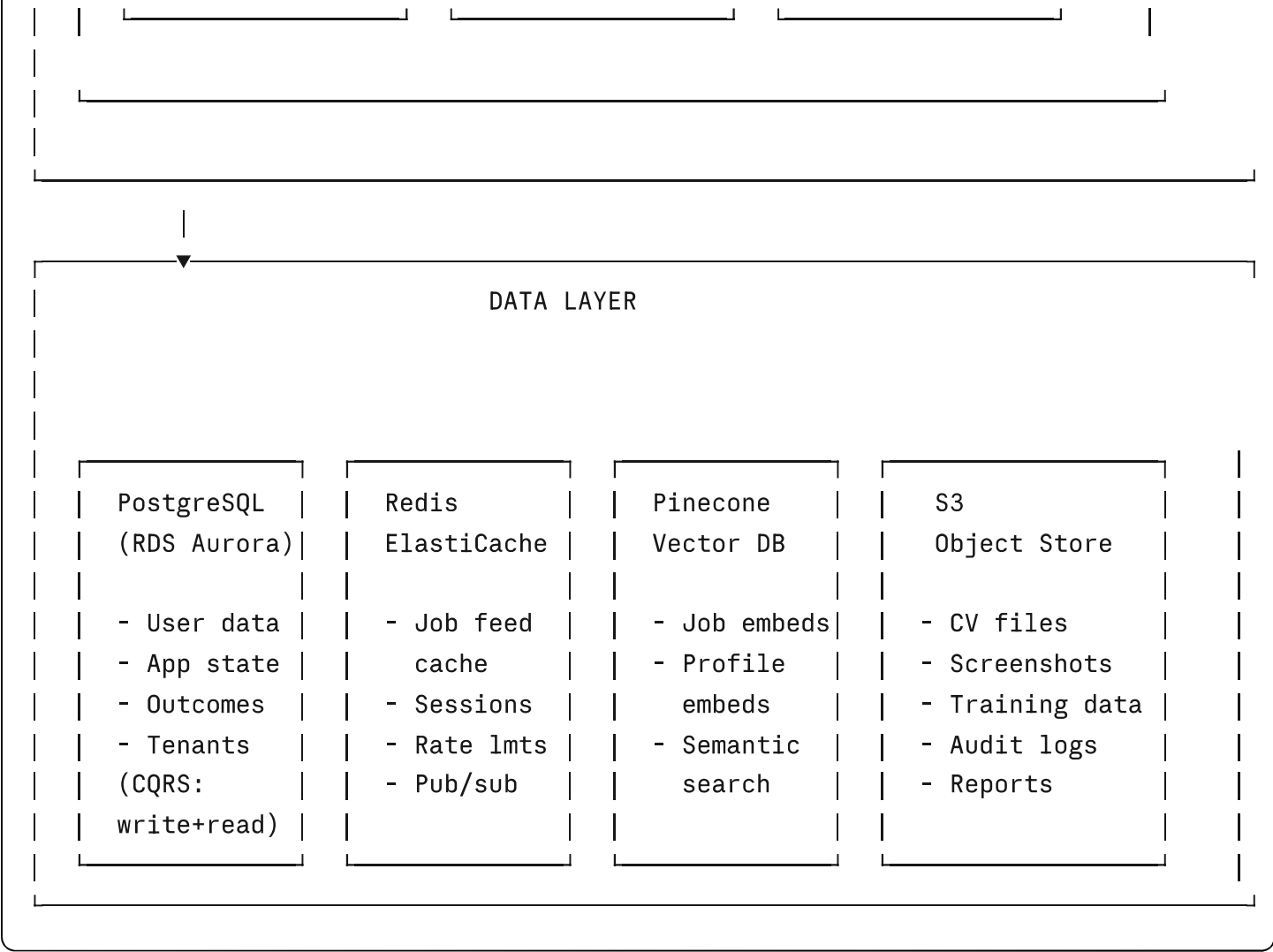
(Claude/GPT-4o)

(weekly batch)

500+ parallel

Batched calls

Outcome ranker



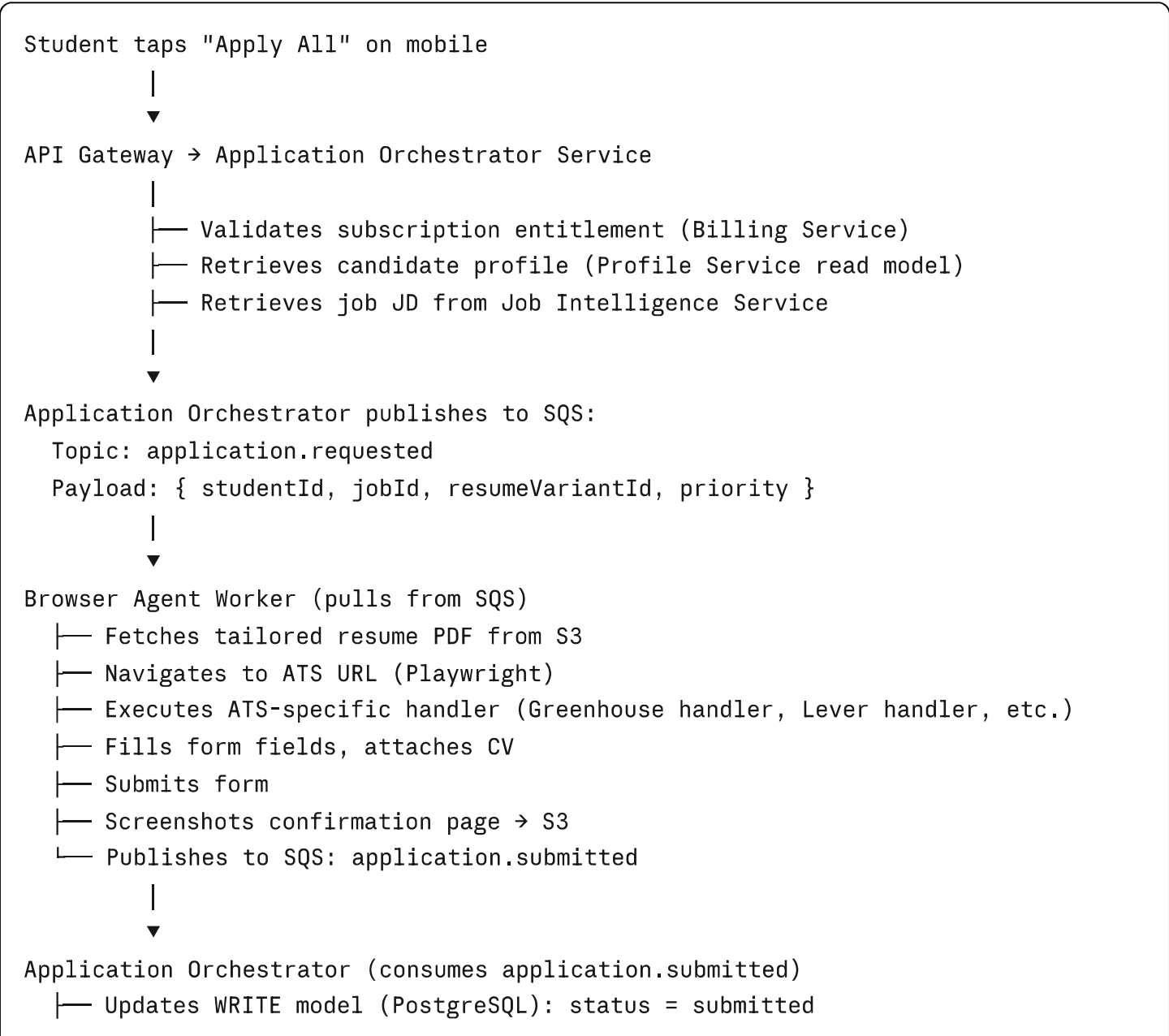
4.2 Key Architectural Patterns

Pattern	Where Applied	Rationale
Microservices	All core domains	Independent deployability; team autonomy; fault isolation
CQRS	Application Engine + Job Feed	Write path (submissions) separated from read path (dashboard) for scale
Event-Driven	Application Orchestrator → Worker Fleet	Decouples browser automation from API response; enables retries
Serverless	Webhook ingestion, CV parsing, scheduled digests	Spiky, stateless workloads; pay-per-invocation
Message Queues	SQS between orchestrator and workers	Backpressure management; durable delivery; dead-letter handling

Pattern	Where Applied	Rationale
Edge Caching	Job intelligence feed via CloudFront	Job feeds change hourly; 95% of reads can be cache-served
Vector Search	Job-candidate semantic matching	Pinecone enables sub-50ms semantic matching at 1M+ embeddings
Multi-tenancy	Row-level security in PostgreSQL; tenant-scoped JWT claims	Cost-efficient B2B isolation without per-tenant database clusters

5. Component Design & Data Flow

5.1 Critical Path: Student Submits Application



- | Publishes to WebSocket channel (Redis pub/sub → Notification Service)
- | Triggers LLM Pipeline Worker: generate outreach message



Student dashboard updates in real-time (< 2s from confirmation)

5.2 Critical Path: Outcome Learning Feedback Loop

Employer emails or ATS webhook fires (interview invite)



Webhook Ingestion Lambda (AWS Lambda)

- | Validates webhook signature
- | Parses event type (interview_invite | rejection | offer)
- | Publishes to SQS: outcome.received



Application Orchestrator (consumes outcome.received)

- | Updates application record: status = interview_invited
- | Publishes to SNS: outcome.recorded (fan-out)
 - | | Notification Service → Push notification to student
 - | | Interview AI Service → Pre-generate company brief
 - | | Outcome Event Store (append-only S3 Parquet + Glue catalog)



Weekly ML Training Batch (EventBridge → Step Functions)

- | Reads outcome events from S3/Glue
- | Joins with: job_description, resume_version, cover_letter, ATS_platform
- | Trains/fine-tunes job-matching ranker (SageMaker Training Job)
- | Evaluates vs holdout set (AUC > 0.70 gate)
- | Promotes model to production endpoint (SageMaker Model Registry)

5.3 Multi-Tenant University B2B Data Flow

University Student (SSO login)



Identity Service → SAML/OIDC assertion from university IdP

- | Maps external identity to platform userId
- | Attaches tenant_id claim: "university:oxford-2026"
- | Issues JWT with claims: { sub, tenant_id, role: "student", tier: "institutional" }



All subsequent API calls carry `tenant_id` in JWT



Every database query appended with:

`WHERE tenant_id = :tenant_id` (enforced by row-level security policy)

University Admin requests cohort report:

- Analytics Service reads from READ replica
- Row-level security filters to `tenant_id` scope only
- Report generated as PDF → S3 (tenant-scoped bucket prefix)
- Pre-signed URL returned to admin (15-minute expiry)

6. Technology Stack

6.1 Frontend

Component	Technology	Justification
Web application	Next.js 14 (App Router)	SSR for SEO (career content marketing); React Server Components reduce client bundle; Vercel deployment aligns with existing team skill
Mobile	React Native (Expo)	Code sharing with web logic layer; Expo EAS for OTA updates without app store review cycles
PWA	Next.js + Service Worker	Offline job feed browsing; background sync for application status; eliminates mandatory app install for students
UI components	shadcn/ui + Tailwind CSS	Unstyled primitives; avoids library lock-in; Tailwind compile-time purging keeps bundle small
Real-time updates	WebSocket (API Gateway WebSocket) + React Query	Application status updates; interview session signaling; React Query handles polling fallback
State management	Zustand + React Query	Zustand for UI state; React Query for server state/caching; avoids Redux boilerplate
CV/document rendering	react-pdf	In-browser PDF preview of tailored resume before submission

6.2 Backend Services

Service	Runtime	Framework	Justification
All microservices	Node.js 22	tRPC + Fastify	tRPC provides end-to-end type safety eliminating an entire class of contract bugs; Fastify outperforms Express 2–3× on throughput
Browser Agent Workers	Node.js 22	Playwright	Most mature, best-maintained browser automation; supports Chromium/Firefox/WebKit; built-in network interception for fingerprint randomisation
ML Training Pipeline	Python 3.12	SageMaker SDK + scikit-learn/XGBoost	Python is non-negotiable for ML; XGBoost for structured tabular outcome data (outperforms neural nets for this task size)
CV Parsing Lambda	Python 3.12	PyMuPDF + spaCy	Lambda for spiky invocations; PyMuPDF fastest PDF parser; spaCy for NER (skills, companies, dates)
ORM	Prisma	—	Type-safe; migration tooling; compatible with Aurora Postgres
Job Scheduling	BullMQ (Redis-backed)	—	Distributed queue with priority, retries, rate limiting; replaces cron for application scheduling
Background jobs	Trigger.dev	—	Observability into background job runs; dashboard for debugging failed applications

6.3 Databases

Store	Technology	Use Case	Justification
Primary relational	Amazon Aurora PostgreSQL (Serverless v2)	Users, applications, outcomes, tenants	Serverless v2 scales compute to zero between peaks (critical for cost at student pricing); Aurora Global Database for DR
Read replicas	Aurora Read Replica (×2)	Dashboard queries, reporting, analytics	CQRS read model; separates analytical from transactional load

Store	Technology	Use Case	Justification
Cache	Amazon ElastiCache for Redis (Cluster mode)	Job feed cache, sessions, rate limiting, pub/sub	Redis Cluster for horizontal scale; sub-millisecond latency for feed serving
Vector search	Pinecone (Serverless)	Semantic job-candidate matching	Managed; serverless billing; 1M+ vector scale; sub-50ms query; no operational overhead
Object storage	Amazon S3	CVs, screenshots, training data, reports	Lifecycle policies (move to Glacier after 90 days); server-side encryption; WORM for audit logs
Event store	S3 + AWS Glue + Athena	Outcome events, analytics	Serverless data lake pattern; Athena for ad-hoc SQL on Parquet; avoids cost of dedicated data warehouse at early scale
Search	Elasticsearch (OpenSearch Serverless)	Full-text job search, employer search	Only activated for text search; Pinecone handles semantic; separation keeps each optimal

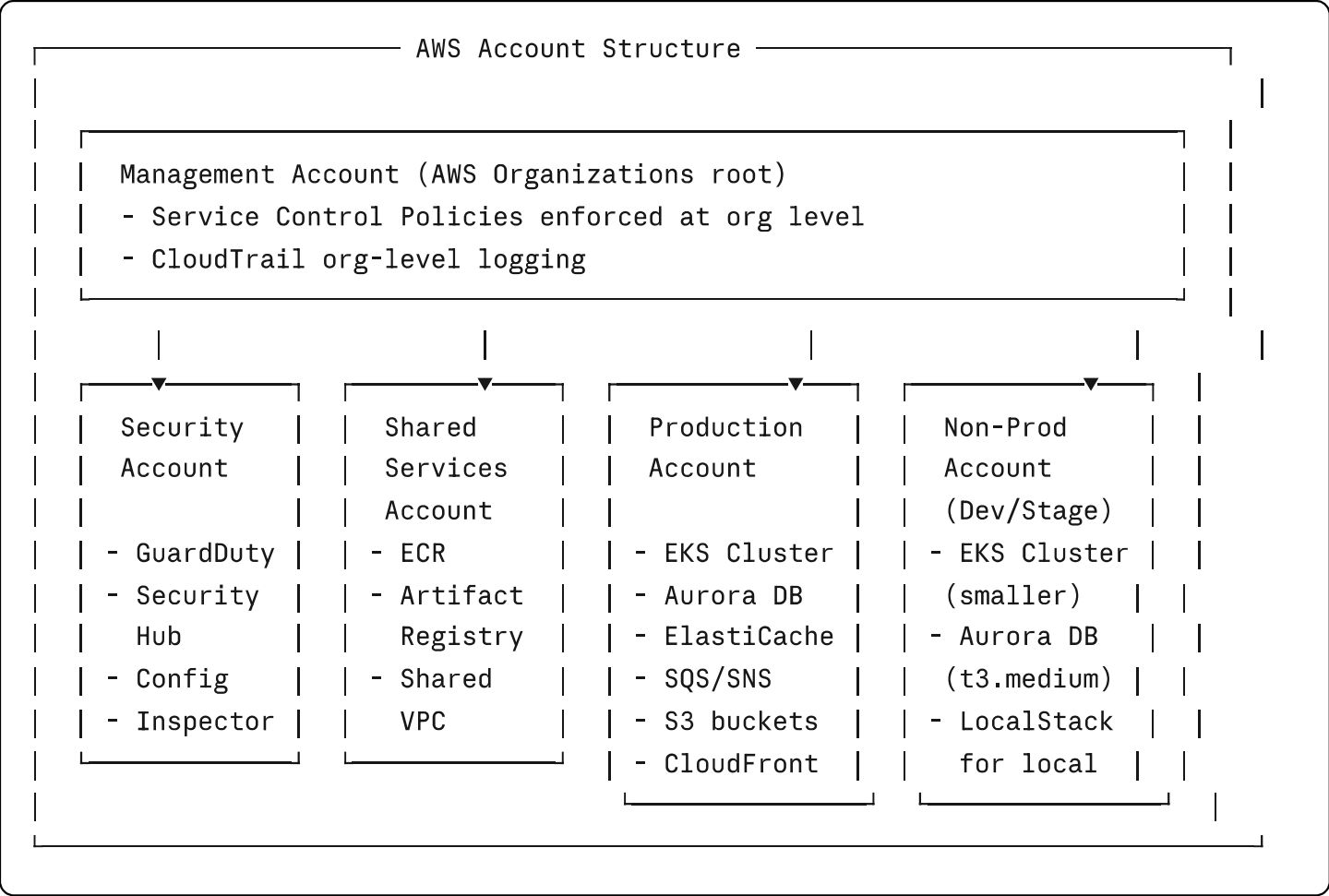
6.4 AI/ML Stack

Component	Technology	Justification
Primary LLM	Anthropic Claude Sonnet 4	Superior structured output (JSON mode); best-in-class at resume tailoring and cover letter generation; lower hallucination rate on factual tasks
LLM fallback	OpenAI GPT-4o	Vendor resilience; automatic failover if Anthropic API degrades; prevents single-vendor outage
LLM orchestration	LangChain (Python)	Prompt chaining, caching, structured output parsing; integrates with both providers
Prompt caching	Anthropic prompt caching + Redis	Cache job description embeddings and system prompts; reduces token cost 60–70% for repeat calls
Embeddings	OpenAI text-embedding-3-large	Best-in-class at 1536 dimensions; stored in Pinecone
ML training	Amazon SageMaker	Managed training; model registry; A/B testing endpoints; integrates with Step Functions

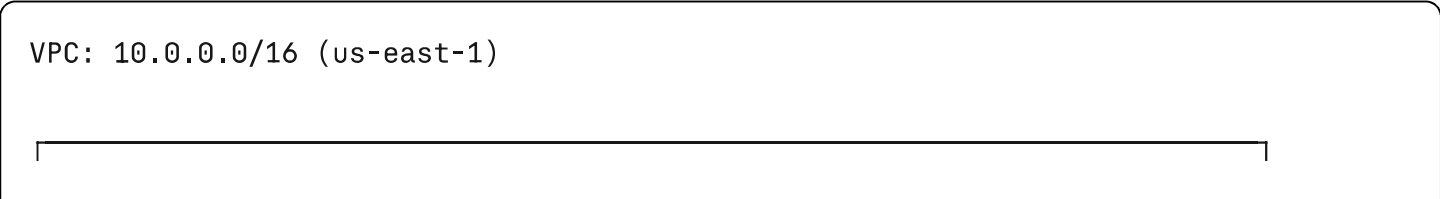
Component	Technology	Justification
Voice STT	Deepgram Nova-2	Real-time, low-latency transcription; student voice interview sessions
Voice TTS	ElevenLabs	Highest naturalness for AI interviewer voice; latency acceptable for turn-based interview sessions
OCR (CV parse)	AWS Textract	Managed OCR; handles multi-column layouts common in academic CVs

7. Cloud Infrastructure & Networking

7.1 AWS Infrastructure Overview



7.2 VPC & Network Design (Production)



Public Subnets (10.0.1.0/24, 10.0.2.0/24, 10.0.3.0/24 – 3 AZs)

Internet Gateway → ALB (Application Load Balancer)

NAT Gateway (one per AZ for HA)

Private App Subnets (10.0.10.0/24, /24, /24 – 3 AZs)

EKS Worker Nodes (m7g.2xlarge for app services)

EKS Worker Nodes (c6i.4xlarge for browser agent workers)

Private Data Subnets (10.0.20.0/24, /24, /24 – 3 AZs)

Aurora PostgreSQL (Multi-AZ)

ElastiCache Redis Cluster

SageMaker Training (VPC mode)

↕ VPC Peering / Transit Gateway

VPC: 10.1.0.0/16 (eu-west-2 – UK secondary region, DR + data residency)

7.3 Kubernetes (EKS) Architecture

EKS Cluster: launchvector-prod

Node Groups:

app-services-ng	m7g.2xlarge (Graviton3)	3-20 nodes
-----------------	-------------------------	------------

- All microservices	Spot + On-demand mix	HPA + KEDA
---------------------	----------------------	------------

browser-workers-ng	c6i.4xlarge	5-100 nodes
--------------------	-------------	-------------

- Playwright agents	Spot Instances (70%)	KEDA (SQS)
---------------------	----------------------	------------

- Chromium headless	On-demand (30% base)	Queue depth
---------------------	----------------------	-------------

ml-training-ng	p3.2xlarge (GPU)	0-5 nodes
----------------	------------------	-----------

- Weekly batch only	On-demand; scale to 0	EventBridge
---------------------	-----------------------	-------------

- SageMaker offload	between training runs	triggered
---------------------	-----------------------	-----------

Cluster Add-ons:

- AWS Load Balancer Controller (ALB Ingress)

- Cluster Autoscaler

- KEDA (Kubernetes Event-Driven Autoscaling) – scales on SQS depth
- Cert-Manager (Let's Encrypt / ACM)
- External Secrets Operator (AWS Secrets Manager → K8s secrets)
- Datadog Agent (DaemonSet)
- Karpenter (next-gen node provisioner – faster than CA)

7.4 CDN & Global Distribution

Layer	Technology	Configuration
CDN	CloudFront	Origins: S3 (static assets), ALB (API), Vercel (Next.js)
Static assets	S3 + CloudFront	Immutable hashing; 1-year cache TTL; Brotli compression
API responses	CloudFront + API Gateway	Job feed: 5-min TTL; Profile: no-cache; Application status: no-cache
Edge compute	Lambda@Edge	JWT validation at edge; geo-routing; A/B test assignment
DNS	Route 53	Latency-based routing; health checks; failover to eu-west-2
DDoS	AWS Shield Standard (upgraded to Advanced at scale)	Automatic L3/L4 protection; CloudFront integration

8. Security Architecture

8.1 Identity & Access Management

Authentication Flow:

B2C Student:

- Signup → AWS Cognito User Pool
 - Email verification
 - MFA (TOTP optional, SMS optional)
 - Issues JWT (access + refresh tokens)
 - Access token: 1-hour expiry
 - Refresh token: 30-day expiry, rotated on use

B2B University (SSO):

- Login → University IdP (Shibboleth / Azure AD)
 - SAML 2.0 assertion to Cognito Identity Pool
 - Cognito maps to federated identity
 - JWT issued with tenant_id + institutional role
 - Same token format as B2C (transparent to services)

API Authorization:

- Every request → API Gateway Lambda Authorizer
 - Validates JWT signature (Cognito JWKS)
 - Extracts: userId, tenant_id, tier, role
 - Passes context to downstream service
 - Services trust authorizer output (no re-validation)

8.2 Zero-Trust Architecture

Principle	Implementation
Never trust, always verify	mTLS between all microservices (Istio service mesh)
Least privilege	IAM roles per EKS service account (IRSA); no shared credentials
Assume breach	Workload isolation via Kubernetes NetworkPolicies; no service can reach DB without explicit allow
Verify explicitly	Every internal API call carries JWT; no implicit trust from network location
Micro-segmentation	Security Groups at pod level (VPC CNI); each service in own namespace with NetworkPolicy

8.3 Secrets Management

Secret Lifecycle:

- Developer/CI creates secret → AWS Secrets Manager
- External Secrets Operator (K8s) → Syncs to K8s Secret (in-cluster)
- Pod mounts K8s Secret as env var or volume
- Secret rotation → Secrets Manager auto-rotates DB passwords (30-day)
 - ESO re-syncs within 60s
 - Zero downtime rotation via Aurora multi-user

What is stored where:

AWS Secrets Manager:

- Database credentials (Aurora master + app users)
- Anthropic API key, OpenAI API key
- Stripe secret key, webhook signing secret
- Deepgram API key, ElevenLabs API key
- Pinecone API key
- Third-party OAuth secrets

AWS Parameter Store (SSM):

- Non-secret config (feature flags, tier limits)
- Environment-specific settings

K8s Secrets (synced from Secrets Manager):

- Mounted at pod runtime only
- Never committed to Git
- Never logged

8.4 Data Security

Layer	Control	Implementation
At-rest encryption	AES-256	Aurora: AWS KMS CMK; S3: SSE-S3; ElastiCache: in-transit + at-rest
In-transit encryption	TLS 1.3	ALB → services; services → DB; services → external APIs
CV file access	Pre-signed URLs	S3 pre-signed URLs (15-min expiry); no public bucket ACLs
PII tokenisation	Field-level	Student email, phone hashed with HMAC for analytics; plaintext only in Profile Service
GDPR right to erasure	Soft delete → hard delete pipeline	30-day soft delete → automated hard delete + S3 purge
Audit logging	Append-only S3 with Object Lock	All data access events; WORM compliance for FERPA
Database encryption	Column-level (pgcrypto)	PII fields encrypted at column level (belt-and-suspenders beyond disk encryption)

8.5 Network Security

Layer	Control
WAF	AWS WAF on CloudFront + ALB: OWASP Top 10 ruleset; rate limiting (100 req/min per IP)
DDoS	AWS Shield Standard; CloudFront absorbs L7 floods
Rate limiting	API Gateway: per-user (20 req/s), per-tenant (500 req/s), per-IP (100 req/min)
Egress control	NAT Gateway with restricted egress SG; only whitelisted external endpoints
VPC Endpoints	S3, SQS, SNS, Secrets Manager, ECR, CloudWatch — all via VPC endpoints (no internet transit)
Vulnerability scanning	Amazon Inspector (container images + EC2); Snyk in CI pipeline
Penetration testing	Annual external pentest + OWASP ZAP in CI for DAST

9. Data Architecture & Storage

9.1 PostgreSQL Schema Design (Key Tables)

```
sql
```

```

-- Multi-tenancy enforced via RLS policy on all tables
CREATE POLICY tenant_isolation ON users
  USING (tenant_id = current_setting('app.tenant_id')::uuid);

-- Core entities (abbreviated for clarity)
users (
  id UUID PK,
  tenant_id UUID FK → tenants(id),
  email_hash BYTEA,          -- HMAC of email (PII protection)
  email_encrypted BYTEA,     -- pgcrypto encrypted
  subscription_tier TEXT,    -- 'grad' | 'starter' | 'professional' | 'institutional'
  created_at TIMESTAMPTZ,
  deleted_at TIMESTAMPTZ    -- soft delete
)

candidate_profiles (
  id UUID PK,
  user_id UUID FK,
  tenant_id UUID,
  raw_cv_s3_key TEXT,        -- path to original CV in S3
  structured_profile JSONB,  -- extracted skills, experience, education
  embedding_vector_id TEXT,  -- Pinecone vector ID
  target_roles TEXT[],
  salary_floor INTEGER,
  max_commute_minutes INTEGER,
  updated_at TIMESTAMPTZ
)

applications (              -- WRITE model
  id UUID PK,
  user_id UUID FK,
  tenant_id UUID,
  job_id UUID FK,
  resume_version_id UUID FK,
  ats_platform TEXT,
  status TEXT,              -- queued|submitting|submitted|viewed|response|interview
  submitted_at TIMESTAMPTZ,
  response_at TIMESTAMPTZ,
  screenshot_s3_key TEXT,
  outcome TEXT              -- null until known
)

application_dashboard_view ( -- READ model (materialised view, refreshed async)

```



```

-- Denormalised for fast dashboard queries; updated via event
)

job_outcomes (
    -- Append-only outcome event store (partitioned by month)
    id UUID PK,
    application_id UUID,
    user_id UUID,
    job_id UUID,
    outcome_type TEXT,          -- interview_invite | rejection | offer | no_response
    days_to_response INTEGER,
    resume_version_id UUID,
    cover_letter_hash TEXT,
    ats_platform TEXT,
    recorded_at TIMESTAMPTZ
) PARTITION BY RANGE (recorded_at);

```

9.2 CQRS Implementation

Write Side (Application Orchestrator → Aurora Primary):

- All mutations: submit_application, record_outcome, update_status
- Synchronous write; immediate consistency
- Publishes domain events to SQS after commit

Read Side (Dashboard Service → Aurora Read Replica):

- Materialised views refreshed by event consumer
- application_dashboard_view: denormalised per-student pipeline
- job_match_view: pre-computed match scores cached in Redis
- Eventual consistency acceptable (< 2s lag)

Event Consumer (Lambda):

- Consumes SQS events
- Updates materialised views on read replica
- Invalidates Redis cache keys
- Triggers WebSocket push via Notification Service

9.3 Data Residency & Compliance

Data Category	Storage Location	Legal Basis
EU/UK student PII	eu-west-2 (Aurora + S3)	GDPR Art. 6(b) — contract performance
US student PII	us-east-1	FERPA (institutional), CCPA (California)

Data Category	Storage Location	Legal Basis
Application outcomes (analytics)	us-east-1 (anonymised)	Legitimate interest; pseudonymised
CV files	Region-matched to user	GDPR data minimisation
Audit logs	Both regions (replicated)	GDPR Art. 30 records; FERPA audit

10. API Design & Integration Layer

10.1 API Architecture

External API Surface:

REST API (AWS API Gateway):

- /api/v1/auth/* → Identity Service
- /api/v1/profile/* → Profile Service
- /api/v1/jobs/* → Job Intelligence Service
- /api/v1/applications/* → Application Orchestrator
- /api/v1/outreach/* → Outreach Service
- /api/v1/interview/* → Interview AI Service
- /api/v1/billing/* → Billing Service
- /api/v1/admin/* → University Admin (B2B)

WebSocket API (API Gateway WebSocket):

- \$connect / \$disconnect → Managed by Notification Service
- application.status.{userId} → Real-time status push
- interview.session.{sessionId} → Voice interview signaling

Internal APIs (tRPC, mTLS):

- Service-to-service calls
- No public exposure
- Type-safe contracts

10.2 API Design Standards

Standard	Implementation
Versioning	URI versioning (/v1/, /v2/) — never break existing clients
Pagination	Cursor-based (opaque cursor) for all list endpoints; 20 items default, 100 max
Error format	RFC 7807 Problem Details JSON

Standard	Implementation
Rate limiting	Response headers: X-RateLimit-Limit, X-RateLimit-Remaining, X-RateLimit-Reset
Authentication	Bearer token (JWT) in Authorization header; never in URL
Idempotency	POST endpoints accept Idempotency-Key header; stored in Redis 24h
Content type	application/json; UTF-8; gzip content encoding
Deprecation	Sunset header with 6-month notice period

10.3 ATS Integration Layer

ATS Handler Registry Pattern:

```

abstract class ATSHandler {
    abstract async fillForm(page: Page, profile: CandidateProfile, job: Job):
Promise<void>
    abstract async submitForm(page: Page): Promise<string> // Returns confirmation
URL
    abstract get platform(): string
    abstract get successSelector(): string
}

class GreenhouseHandler extends ATSHandler { ... }
class LeverHandler extends ATSHandler { ... }
class WorkdayHandler extends ATSHandler { ... }
class AshbyHandler extends ATSHandler { ... }
class RipplingHandler extends ATSHandler { ... }

```

ATSHandlerRegistry:

- Keyed by ATS platform identifier
- Detected from job URL pattern matching
- Fallback: GenericFormHandler (heuristic-based)

Fingerprint Randomisation:

- Random viewport size (1280-1920 width)
- Random user agent rotation (pool of 50 real browser UAs)
- Random typing speed (80-140 WPM simulation)
- Random mouse movement paths
- Residential proxy rotation (Bright Data) for blocked platforms

11. AI/ML Pipeline Architecture

11.1 LLM Pipeline Design

Resume Tailoring Pipeline (per application):

Input:

- Candidate structured profile (JSONB from DB)
- Job description (fetched + cleaned)
- Target tone (conservative | neutral | assertive)

Step 1: JD Analysis (Claude Sonnet 4)

System prompt: [cached – ATS keyword extraction prompt]

User: {job_description}

Output: { required_skills[], nice_to_have[], keywords[], seniority_signals }

Cache: Redis key = hash(job_description) → 24h TTL

Step 2: Resume Section Selection (rule-based, no LLM)

Match candidate skills to required_skills

Rank experience bullets by relevance score (XGBoost ranker)

Select top N bullets within 1-page constraint

Step 3: Resume Rewriting (Claude Sonnet 4)

System prompt: [cached – resume tailoring persona]

User: {selected_bullets} + {target_keywords} + {job_context}

Output: Rewritten bullets, adjusted summary

Max tokens: 800 (cost control)

Step 4: Cover Letter Generation (Claude Sonnet 4)

System prompt: [cached – cover letter persona]

User: {tailored_resume_summary} + {company_context} + {job_context}

Output: 250-word cover letter

Max tokens: 400 (cost control)

Cost per application (estimated):

- JD analysis: ~\$0.003 (cached after first use per job)
- Resume rewrite: ~\$0.008
- Cover letter: ~\$0.005
- Total: ~\$0.016/application
- At 20 applications/week × 4 weeks = \$1.28/student/month
- As % of \$29 ARPU = 4.4% (well within 30% target)

11.2 Outcome-Learning ML Pipeline

Weekly Training Batch (EventBridge → Step Functions):

Step 1: Data Extraction (Glue Job)

- Read Parquet outcome events from S3 (past 90 days)
- Join with job_descriptions, resume_versions, ats_platforms
- Produce feature matrix: 180+ features
- Train/validation split: 80/20, stratified by outcome type

Step 2: Feature Engineering

- TF-IDF overlap: resume keywords n JD keywords
- Embedding similarity: Cosine(profile_embedding, jd_embedding)
- Historical signal: employer response rate (from prior applications)
- ATS platform: one-hot encoded (platform-specific pass rates)
- Application timing: hours since posting (recency signal)

Step 3: Model Training (SageMaker XGBoost)

- Target: binary – did application generate interview_invite?
- Algorithm: XGBoost (gradient boosting; handles tabular data well)
- Hyperparameter tuning: SageMaker Automatic Model Tuning
- Training time: ~45 min on ml.m5.4xlarge

Step 4: Evaluation Gate

- AUC-ROC > 0.70 required for promotion
- Precision@K > 0.15 (top K jobs should yield interviews)
- If gate fails → alert; keep previous model

Step 5: Model Registration & Deployment

- SageMaker Model Registry → version tracking
- Blue/green deployment to SageMaker Endpoint
- Shadow mode: new model runs in parallel for 1 week before cutover
- Rollback: single API call if production metrics degrade

11.3 Voice Interview AI Architecture

Interview Session Flow:

Student initiates mock interview:

- Interview AI Service creates session
- Issues session token (JWT, 90-min expiry)
- Returns WebSocket URL for voice streaming

Voice pipeline:

- Student microphone → WebRTC (browser/native)
 - Interview AI Service (WebSocket)
 - Deepgram Nova-2 STT (streaming)
 - Transcript segment (real-time)
 - Claude Sonnet 4 (response generation)
 - ElevenLabs TTS (voice synthesis)
 - Audio stream back to student

STAR Evaluation (post-answer):

- Transcript stored per answer
- Claude evaluates: Situation/Task/Action/Result presence
- Scores: structure (0-10), specificity (0-10), confidence signals
- Feedback generated and queued for end-of-session report

Session storage:

- Transcript: S3 (student-owned; deletable on request)
- Scores: PostgreSQL (for dashboard history)
- Audio: NOT stored (privacy by design)

12. Scalability & Performance

12.1 Scaling Strategy by Component

Component	Scaling Mechanism	Trigger	Limits
API Gateway	Managed (AWS)	Automatic	10K req/s default; 29K with increase
Microservices (EKS)	HPA (CPU/memory) + KEDA	CPU > 60%; queue depth	3 min → 20 instances
Browser Workers	KEDA (SQS depth)	Queue depth > 50 jobs	5 → 100 worker pods
Aurora PostgreSQL	Serverless v2 (ACU scaling)	Connection count; CPU	0.5 → 128 ACUs
Read Replicas	Manual (add replica)	Read latency > 50ms	Up to 15 Aurora replicas
ElastiCache Redis	Cluster mode (shard scaling)	Memory > 70%	Add shards; vertical resize

Component	Scaling Mechanism	Trigger	Limits
SQS	Managed; unlimited	N/A	Effectively unlimited throughput
S3	Managed; unlimited	N/A	No scaling required
Pinecone	Serverless (auto-scale)	Query volume	Managed by Pinecone

12.2 Caching Architecture

Cache Hierarchy:

L1 – Browser/Client Cache:

- Static assets: 1-year immutable cache
- API responses: ETag-based conditional requests

L2 – CloudFront Edge Cache:

- Job feed (top 1000 results): 5-min TTL
- Company research briefs: 24-hour TTL
- Salary data: 1-hour TTL

L3 – Redis Application Cache:

- Job match scores per student: 15-min TTL (invalidated on profile update)
- LLM prompt outputs (JD analysis): 24-hour TTL keyed on JD hash
- Session data: 1-hour sliding TTL
- Rate limit counters: 1-min sliding window

L4 – Database Query Cache:

- Aurora query cache (read replicas)
- Materialised views refreshed on events

LLM-specific caching (cost-critical):

- Anthropic prompt caching: system prompts cached 5 min server-side
- Reduces input token cost by ~60% for repeat patterns
- JD analysis results: Redis 24h (same JD submitted by multiple students)

12.3 Performance Optimisation

Area	Technique	Target Improvement
Job feed	Pre-computed, cached ranked lists	500ms → <50ms

Area	Technique	Target Improvement
Resume tailoring	Parallel LLM calls (Promise.all)	3× throughput increase
CV parsing	Lambda warm pool (provisioned concurrency)	Cold start eliminated
DB queries	Covering indexes on tenant_id + user_id + status	10ms → <2ms
ATS submissions	Worker pooling (pre-authenticated browser sessions)	30s → <10s setup overhead
Mobile app	React Native Hermes engine; bundle splitting	40% smaller JS bundle
API responses	Brotli compression (75% smaller than gzip on JSON)	75% bandwidth reduction

13. Reliability, Fault Tolerance & DR

13.1 Availability Architecture

Multi-AZ Deployment (Production):

us-east-1 (Primary):

AZ-1a: EKS worker nodes, Aurora primary, Redis primary

AZ-1b: EKS worker nodes, Aurora standby, Redis replica

AZ-1c: EKS worker nodes, Aurora replica (read), Redis replica

Automatic failover:

- Aurora: 30-60s RDS-managed failover to standby
- Redis: ElastiCache automatic primary election
- EKS pods: rescheduled to healthy AZ within 60s
- ALB: health checks; routes around unhealthy instances

Cross-region (eu-west-2 for DR + UK data residency):

- Aurora Global Database: <1s replication lag
- S3 Cross-Region Replication: async (eventual consistency)
- DNS failover: Route 53 health check → failover record (< 60s)
- RTO: 45 minutes (manual promotion of Aurora secondary)
- RPO: < 1 minute (Aurora Global Database lag)

13.2 Resilience Patterns

Pattern	Implementation	Applied To
Circuit Breaker	node-circuit-breaker in each service	Outbound LLM API calls; ATS submissions
Retry with backoff	Exponential backoff + jitter (BullMQ)	SQS consumers; external API calls
Dead Letter Queue	SQS DLQ (3 retries → DLQ)	Failed application jobs; webhook processing
Timeout budgets	Per-service timeouts (LLM: 30s, ATS: 5min)	All external calls
Bulkhead	Separate worker pools per ATS platform	Workday issues don't starve Greenhouse
Graceful degradation	Job feed served from cache if DB unreachable	Job Intelligence Service
Idempotency	Redis-based idempotency keys	All application submissions (prevent duplicates)
Health checks	/healthz (liveness) + /readyz (readiness)	All K8s pods; ALB target groups

13.3 Disaster Recovery Runbook Summary

Scenario	Detection	Response	RTO
Single AZ failure	CloudWatch AZ health	Auto (multi-AZ)	< 2 min
Aurora primary failure	RDS event + CloudWatch	Auto failover	< 2 min
EKS control plane disruption	EKS health dashboard	AWS-managed; manual pod restart if needed	< 10 min
Full region failure (us-east-1)	Route 53 health check fails	Manual: promote Aurora Global DB secondary; update DNS	< 45 min

Scenario	Detection	Response	RTO
LLM provider (Anthropic) outage	Circuit breaker trips	Auto: failover to OpenAI GPT-4o	< 30 sec
Breach / data exfiltration	GuardDuty alert	Automated: isolate namespace; revoke credentials; notify DPO	15 min isolation

14. Observability & Monitoring

14.1 Three Pillars Architecture

Metrics → Amazon CloudWatch + Datadog

- Infrastructure: CPU, memory, disk, network per pod
- Application: request rate, error rate, latency (p50/p95/p99)
- Business: applications submitted/hour, LLM cost/user, conversion rate
- Custom: ATS success rate per platform, interview invite rate

Logs → CloudWatch Logs + Datadog Log Management

- Structured JSON logging (all services)
- Log levels: INFO (business events), WARN (degradation), ERROR (failures)
- PII scrubbing middleware (strips email, name before logging)
- Log retention: 30 days hot, 1 year cold (S3 Glacier)

Traces → AWS X-Ray + Datadog APM

- Distributed tracing across all microservices
- Trace sampling: 100% for errors, 5% for success paths
- Critical paths traced end-to-end: resume tailoring, ATS submission
- Service map: visual dependency graph (used for incident response)

14.2 Alerting Strategy

Alert	Threshold	Channel	Severity
API error rate	> 1% over 5 min	PagerDuty	P1
Application submission failure rate	> 5% over 15 min	PagerDuty	P1
Aurora CPU	> 80% sustained 10 min	Slack + PagerDuty	P2
LLM API cost	> 120% of daily budget	Slack	P2

Alert	Threshold	Channel	Severity
SQS queue depth (browser workers)	> 5,000 jobs	Slack	P2
ATS platform block detected	> 50% failure rate on one platform	Slack	P2
Student NPS drop	Survey score < 40 (weekly digest)	Email	P3
Cert expiry	TLS cert expires in < 30 days	Slack	P3

14.3 Business Intelligence Dashboard

Real-time Metrics (Datadog Dashboard):

- └─ Applications submitted (last 24h)
- └─ Interview invite rate (7-day rolling)
- └─ LLM cost per user (current month)
- └─ ATS success rate by platform
- └─ Active browser workers (vs queue depth)
- └─ New student signups (hourly)

Weekly Automated Report (SES → Founder team):

- └─ Cohort placement rate
- └─ Revenue metrics (MRR, churn, new subs)
- └─ ML model performance (AUC, precision@K)
- └─ Infrastructure cost vs revenue ratio

University Admin Dashboard (B2B):

- └─ Cohort application volume
- └─ Interview invite rate (vs platform average)
- └─ Top employers matched to cohort
- └─ Student engagement (DAU, session duration)

15. CI/CD & DevOps Pipeline

15.1 Pipeline Architecture

Developer Workflow:

```
git push feature-branch
|
```



GitHub Actions – PR Pipeline:

- |— Lint (ESLint, Prettier, Pylint)
- |— Type check (tsc --noEmit, mypy)
- |— Unit tests (Jest + Vitest; coverage > 80%)
- |— Integration tests (Testcontainers – real Postgres, Redis)
- |— Snyk security scan (dependencies + SAST)
- |— Docker build + layer caching
- |— Trivy container scan (CVE check)
- └─ Preview environment deploy (ephemeral per PR)



PR merged to main



GitHub Actions – Deploy Pipeline:

- |— All PR checks (re-run on merge commit)
- |— Build Docker image → push to ECR (sha-tagged)
- |— Integration test suite (against staging)
- |— DAST scan (OWASP ZAP against staging)
- |— Semantic release (auto version bump + changelog)
- |
- |— Deploy to STAGING:
 - | ArgoCD sync → Kubernetes manifest update
 - | Smoke tests (Playwright E2E – 15 critical paths)
 - | Load test (k6 – 500 VU, 5 min)
- |
- └─ Deploy to PRODUCTION (manual approval gate):
 - ArgoCD sync (GitOps – desired state in Git)
 - Progressive rollout: 10% → 30% → 100% (Argo Rollouts)
 - Automated rollback on: error rate > 2% | latency p99 > 2s

15.2 GitOps with ArgoCD

Repository Structure:

- launchvector-platform/ (application code)
- launchvector-infrastructure/ (Terraform – AWS resources)
- launchvector-k8s-manifests/ (Kubernetes desired state – ArgoCD source of truth)
 - |— base/ (shared manifests)
 - |— overlays/
 - | |— development/
 - | |— staging/
 - | └─ production/

Promotion flow:

- Dev: Auto-sync on every commit to main
- Staging: Auto-sync with smoke test gate
- Production: Manual approval → ArgoCD sync → Argo Rollouts progressive delivery

15.3 Infrastructure as Code

Layer	Tool	Rationale
AWS resources	Terraform (Terragrunt for DRY)	Most mature; large module ecosystem; state in S3 + DynamoDB lock
K8s resources	Helm charts + Kustomize overlays	Helm for third-party apps; Kustomize for environment-specific patches
Secret provisioning	Terraform + Secrets Manager	Secrets created via Terraform; values set separately (never in Git)
Database migrations	Prisma Migrate	Checked into Git; run in CI before deployment; never auto-run in prod
AWS accounts	AWS Control Tower	Landing zone governance; SCPs; centralized logging

16. Deployment Strategy & Environment Separation

16.1 Environment Matrix

Attribute	Development	Staging	Production
Deployment trigger	Auto (every commit)	Auto (main branch)	Manual approval
AWS Account	Non-prod account	Non-prod account	Prod account
Database	PostgreSQL (Docker)	Aurora Serverless (small)	Aurora Serverless (full)
LLM	Real APIs (rate-limited)	Real APIs (budget-capped)	Real APIs (full)
Browser agents	LocalStack + mock ATS	Real (5 platforms only)	Real (all platforms)
Data	Anonymised prod sample	Synthetic + anonymised	Live

Attribute	Development	Staging	Production
Monitoring	Local logs only	Full observability	Full + PagerDuty
Cost (monthly)	< \$200	< \$800	Variable (revenue-tied)

16.2 Feature Flag Strategy

LaunchDarkly (feature flag service):

- All major features behind flags
- Rollout strategies:
 - └─ Percentage rollout (5% → 25% → 100%)
 - └─ Segment-based (e.g., beta users only)
 - └─ University-tenant specific (B2B white-labelling)
 - └─ Kill switch (instant disable without deployment)

Flags managed:

- interview_voice_ai: true (Pro tier only)
- outcome_model_v2: 10% canary
- outreach_linkedin: false (disabled pending ToS review)
- grad_scheme_deadlines: true (all tiers)
- university_admin_portal: tenant-scoped

17. Third-Party Integrations

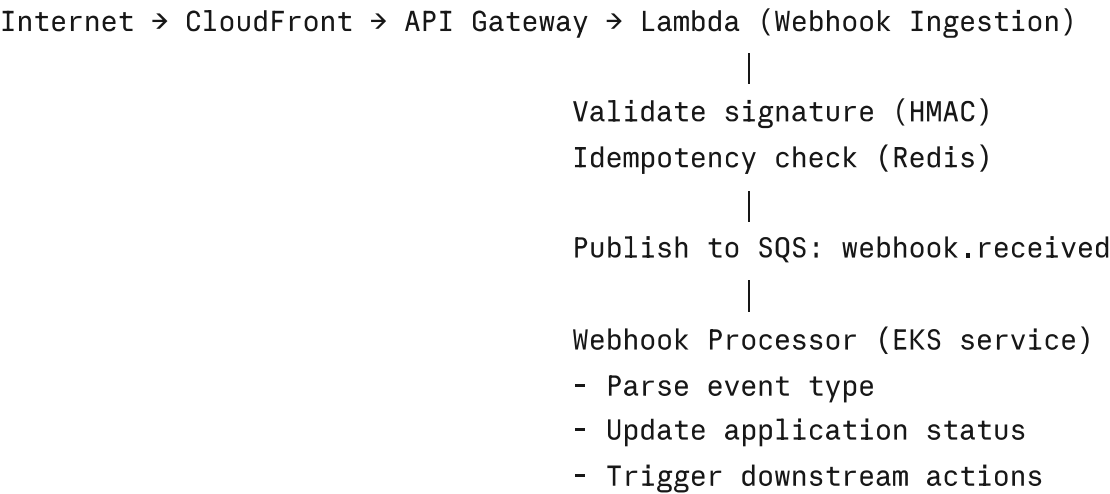
17.1 Integration Inventory

Integration	Purpose	Auth	Failure Mode
Anthropic Claude API	LLM (resume, cover letter, interview)	API key (Secrets Manager)	Failover to OpenAI
OpenAI API	LLM fallback + embeddings	API key	Graceful degradation
Stripe	Subscription billing, metering	Secret key + webhooks	Queue payment events; retry
Deepgram	Voice STT (interview sessions)	API key	Session fails gracefully; text fallback

Integration	Purpose	Auth	Failure Mode
ElevenLabs	Voice TTS (AI interviewer)	API key	Session fails gracefully; text fallback
Pinecone	Vector search (job matching)	API key	Fallback to keyword search (OpenSearch)
Bright Data	Residential proxies (ATS anti-bot)	Account credentials	Rotate to secondary proxy provider
Resend	Transactional email	API key	SES fallback
PostHog	Product analytics (self-hosted option)	API key	Non-critical; degrade silently
LaunchDarkly	Feature flags	SDK key	Default values (fail-open or fail-closed per flag)
Datadog	APM, logs, metrics	API key	CloudWatch fallback for metrics
PagerDuty	On-call alerting	Integration key	SMS fallback to on-call phone
CSCS API (Trades tier)	Card verification	OAuth 2.0	Manual review queue
Shibboleth/Azure AD	University SSO	SAML 2.0 / OIDC	Block login; alert university admin

17.2 Webhook Ingestion Architecture

Inbound webhooks (Stripe, employer ATS callbacks, email tracking):



Webhook reliability:

- SQS provides durable queue (messages retained 14 days)
- Dead-letter queue for failed processing
- Exact-once delivery via idempotency key (Redis, 24h TTL)
- Retry: 3 attempts with exponential backoff

18. Cost Architecture & Optimisation

18.1 Cost Model by Layer

Layer	Cost Driver	Monthly (10K users)	Monthly (50K users)
EKS compute (app services)	EC2 instance hours	\$800	\$3,200
EKS compute (browser workers)	Spot instances (c6i.4xlarge)	\$1,200	\$5,500
Aurora PostgreSQL	ACU-hours (Serverless v2)	\$300	\$900
ElastiCache Redis	r7g.large × 3	\$350	\$700 (+ shards)
LLM APIs (Anthropic + OpenAI)	Token consumption	\$1,280	\$6,400
Pinecone	Serverless (query + storage)	\$200	\$800
S3 + data transfer	Storage + egress	\$150	\$500
CloudFront	Data transfer	\$80	\$300
SageMaker (weekly training)	ml.m5.4xlarge training	\$120	\$120
Third-party SaaS	Stripe, Datadog, LaunchDarkly	\$800	\$1,400
Total infra + AI		~\$5,280	~\$19,820
Revenue	\$29 ARPU	\$290,000	\$1,450,000
Infra as % revenue		1.8%	1.4%

18.2 Cost Optimisation Strategies

Strategy	Implementation	Saving
Spot instances for browser workers	70% Spot / 30% On-demand mix	~60% on compute
Graviton3 for app services	m7g vs m6i equivalent	~20% cheaper
LLM prompt caching	Anthropic caching + Redis	~60% LLM cost reduction
LLM model tiering	Haiku for classification tasks; Sonnet for generation	~40% average LLM saving
Aurora Serverless v2	Scales to 0 in dev/staging	Eliminates idle DB cost
S3 Intelligent-Tiering	Auto-move old CVs to cheaper storage	~70% on aged objects
Reserved Instances (1-year)	Commit on predictable compute (app services)	~30% vs on-demand
Shared read replicas	Pool analytical queries across tenants	Avoid per-tenant replica overhead

19. Architectural Tradeoffs & Risk Register

19.1 Key Architectural Tradeoffs

Decision	Chosen Approach	Alternative Considered	Tradeoff
Multi-tenant DB	Shared DB + RLS	Per-tenant DB	Cost efficiency at the expense of hard isolation. Mitigated by RLS + encryption
Browser automation	Playwright (EKS)	Selenium Grid / Browserbase	EKS gives full control and cost efficiency; managed services (Browserbase) easier to operate but 3-5× cost
Primary LLM	Anthropic Claude	OpenAI only	Claude superior for structured outputs; dual-provider adds operational complexity

Decision	Chosen Approach	Alternative Considered	Tradeoff
Event streaming	SQS + SNS	Kafka (MSK)	SQS is operationally simpler; Kafka only necessary at 10M+ events/day. Re-evaluate at Year 2
Vector DB	Pinecone (managed)	pgvector (self-hosted)	Pinecone at scale outperforms pgvector; managed reduces ops burden. Higher cost acceptable
ML training	SageMaker	Self-managed (on-prem GPU)	SageMaker for speed to production; revisit if training costs exceed \$5K/month
Edge caching	CloudFront	Fastly	CloudFront deep AWS integration; Fastly marginally better performance (future consideration)

19.2 Risk Register

Risk	Probability	Impact	Mitigation
ATS platform UI change breaks handler	HIGH	HIGH	Weekly integration test suite; visual diff on ATS confirmation pages; <24h handler update SLA
Anthropic API rate limit at scale	MED	HIGH	Dedicated rate limits (contact Anthropic enterprise); OpenAI fallback; request queuing
LinkedIn blocks outreach automation	HIGH	MED	Email-primary strategy from launch; LinkedIn de-prioritised; no dependency on LinkedIn API
GDPR enforcement action (EU/UK student data)	LOW	CRITICAL	DPO appointed; DPIA completed; data residency in eu-west-2; Privacy by Design in architecture
LLM-generated content quality degrades	MED	MED	A/B testing on response rates; human-in-loop review for Executive tier; feedback loop from outcome data
Browser fingerprinting detection	MED	HIGH	Residential proxy rotation; randomisation; multiple submission methods per platform
Student churn before placement	HIGH	MED	Natural (placement = success churn); mitigate with referral program; track success vs failure churn separately

Risk	Probability	Impact	Mitigation
Single-region AWS outage	LOW	CRITICAL	Aurora Global Database + DR runbook; eu-west-2 warm standby; 45-min RTO
Cost overrun on LLM at scale	MED	MED	Hard budget caps per API key; automated alerts at 80% budget; model tiering strategy
University SSO integration delays	MED	MED	OAuth2/email fallback during integration; dedicated integration support for enterprise deals

20. Phased Implementation Roadmap

Phase 0 — Foundation (Weeks 1-4)

Priority: Production-ready infrastructure before a single feature ships

Milestone	Work Items	Owner
AWS account structure	Organizations, Control Tower, SCPs, CloudTrail	DevOps
Core networking	VPC, subnets, NAT, VPC endpoints, Route 53	DevOps
EKS cluster (prod + non-prod)	Cluster, node groups, add-ons (KEDA, ALB controller)	DevOps
Aurora PostgreSQL	Serverless v2, multi-AZ, parameter group	Backend
CI/CD pipeline	GitHub Actions, ArgoCD, ECR, Terraform state	DevOps
Secrets management	Secrets Manager, ESO, initial secrets provisioned	DevOps/Security
Identity service	Cognito User Pool, JWT, RBAC, MFA	Backend
Observability stack	Datadog agent, CloudWatch, structured logging	DevOps
Security baseline	WAF rules, GuardDuty, Security Hub, Inspector	Security

Exit criteria: Empty platform deployed to prod with auth, observability, and CI/CD working end-to-end.

Phase 1 — MVP Core (Weeks 5-12)

Priority: Deliver the magic moment — resume upload → 20 applications in 24 hours

Milestone	Work Items	Target
CV parsing pipeline	Textract OCR + Lambda + spaCy NER + S3 storage	Week 6
Profile service	Structured profile storage, gap analysis scoring	Week 6
Job Intelligence MVP	Indeed/LinkedIn job feeds via API, rule-based matching	Week 7
LLM pipeline (resume tailoring)	Claude integration, prompt caching, cover letter gen	Week 8
ATS handlers (5 platforms)	Greenhouse, Lever, Workday basic, Ashby, Rippling	Week 9
Application Orchestrator	Workflow, SQS integration, status tracking	Week 9
Dashboard (real-time)	WebSocket, CQRS read model, mobile-first UI	Week 10
Billing (Stripe)	Subscription setup, entitlement enforcement	Week 10
Beta (50 students)	Closed beta, feedback, iteration	Weeks 11-12

Exit criteria: 50 beta students, 80% submit applications within 24h, >90% ATS success rate, NPS > 40.

Phase 2 — Growth Features (Weeks 13-20)

Priority: Increase placement rates; add outreach; launch publicly

Milestone	Work Items	Target
Hiring manager outreach (email)	Resend integration, outreach AI, follow-up sequences	Week 14
Interview AI (text-based)	Company brief gen, STAR question bank, scoring	Week 15
ATS expansion (20 platforms)	15 additional ATS handlers	Week 16
Job Intelligence ML	Semantic matching with Pinecone, outcome model V0	Week 16
Public launch	ProductHunt, PR, referral program, paid tier enforcement	Week 17
University SSO (B2B)	SAML/OIDC integration, tenant provisioning	Week 18
University admin portal	Cohort dashboard, bulk management, reporting	Week 20
Performance optimisation	Load testing (k6), caching tuning, query optimisation	Week 20

Exit criteria: 500+ paying students, \$74.5K MRR, 20 ATS platforms, first university B2B pilot.

Phase 3 — Intelligence & Scale (Weeks 21–36)

Priority: Outcome learning loop; voice AI; seed fundraise metrics

Milestone	Work Items	Target
Voice interview AI	Deepgram + ElevenLabs + WebRTC pipeline	Week 23
Outcome ML model V1	SageMaker training pipeline, weekly retraining, A/B test	Week 25
Resume A/B testing	Variant tracking, statistical significance gates	Week 26
Hidden job market signals	Funding announcement scraping, job change detection	Week 28
SOC 2 Type II preparation	Controls implementation, audit readiness	Week 30
Multi-region DR validation	DR drill, RTO/RPO validation	Week 30
Horizontal scale test	Load test to 50K concurrent users; Karpenter tuning	Week 32
Series A metrics package	Data room, unit economics report, outcome evidence	Week 36

Exit criteria: \$100K+ MRR, 50+ documented placements, outcome model AUC > 0.70, seed round closed.

Architecture Self-Check

Layer	Covered	Notes
Frontend	✓	Next.js PWA + React Native
Backend services	✓	8 microservices defined
APIs	✓	REST + WebSocket + tRPC internal
Databases	✓	Aurora, Redis, Pinecone, S3, OpenSearch
Auth/authorization	✓	Cognito, JWT, SAML/OIDC, RBAC, RLS
Cloud infrastructure	✓	AWS multi-AZ + eu-west-2 DR
Networking	✓	VPC, subnets, SGs, VPC endpoints
Monitoring/logging	✓	Datadog, CloudWatch, X-Ray
CI/CD pipeline	✓	GitHub Actions + ArgoCD
Security layers	✓	WAF, mTLS, KMS, GuardDuty, zero-trust
Third-party integrations	✓	14 integrations documented
Scalability	✓	HPA, KEDA, Aurora Serverless, Spot
Fault tolerance	✓	Circuit breakers, DLQ, multi-AZ
Disaster recovery	✓	Aurora Global DB, RTO < 45 min
AI/ML pipeline	✓	LLM, voice, outcome learning, embeddings
Cost architecture	✓	Per-layer breakdown, optimisation strategies
Tradeoffs & risks	✓	Risk register with mitigations
Implementation roadmap	✓	3 phases, 36 weeks